

■ Revisão Completa – Banco de Dados NoSQL e MongoDB

1. Conceitos Fundamentais

- Banco Relacional (SQL): usa tabelas, esquema rígido, SQL, garante ACID.
- Banco Não Relacional (NoSQL): orientado a documentos, chave-valor, colunas ou grafos; não exige esquema.
- Cluster: conjunto de servidores que funcionam como um só sistema, garantindo paralelismo, alta disponibilidade.
- Agregado: conjunto de objetos relacionados tratados como uma unidade. É a base para manipulação em lote.

2. Criando e Gerenciando Bancos e Collections no Shell

Criar e usar um banco:
`use Zoologico`

Ver bancos existentes:
`show dbs`

Excluir banco atual:
`db.dropDatabase()`

Criar collection:
`db.createCollection("animais")`

Ver collections:
`show collections`

Excluir collection:
`db.animais.drop()`

3. Documentos e Inserções

Exemplo de documento:

```
{
  nome: "Tucano",
  especie: "ave"
}
```

Inserindo documentos individuais:
`use Zoologico`
`db.animais.insert({ nome: "Tucano", especie: "ave" })`
`db.animais.insert({ nome: "Piranha", especie: "peixe" })`
`db.animais.insert({ nome: "Anta", especie: "mamífero" })`

Inserindo vários documentos de uma só vez:
`db.Emprestimos.insertMany([`
 `{ usuario_id: 1, livros: [ { titulo: "Livro A" }, { titulo: "Livro B" } ], data: "2024-09-01" },`
 `{ usuario_id: 1, livros: [ { titulo: "Livro C" } ], data: "2024-10-01" },`
 `{ usuario_id: 2, livros: [ { titulo: "Livro D" }, { titulo: "Livro E" } ], data: "2024-11-01" }`
`])`

4. Consultas

Buscar todos:
`db.animais.find()`

Formatar saída:
`db.animais.find().pretty()`

Com filtro:
`db.animais.find({ especie: "ave" })`

Com múltiplas condições (AND):
db.animais.find({ especie: "ave", nome: "Tucano" }).pretty()

5. Atualizações e Exclusões

Atualizar campo com \$set:
db.animais.update(
 { especie: "ave" },
 { \$set: { especie: "ave tropical" } }
)

Substituir documento com save():
db.animais.save({ nome: "Dourado Agua Doce", especie: "peixe couro" })

Excluir documento:
db.animais.remove({ especie: "peixe" })

6. Outros Comandos Úteis

Limitar resultados:
db.animais.find().limit(3)

Ordenar resultados:
db.animais.find().sort({ nome: 1 }) // crescente
db.animais.find().sort({ nome: -1 }) // decrescente

Criar índice:
db.animais.ensureIndex({ nome: 1 })

7. Comando Aggregate

O aggregate permite unir coleções, agrupar e projetar dados, como se fosse um JOIN + GROUP BY do SQL

Sintaxe básica:
db.COLLECAO.aggregate([
 { \$match: { /* filtro */ } },
 { \$lookup: { /* junção */ } },
 { \$unwind: "\$campo_array" },
 { \$project: { /* campos a exibir */ } }
)

8. Exemplos de Aggregate – Biblioteca

Associar LIVROS e EMPRESTIMOS:
db.Emprestimo.aggregate([
 {
 \$lookup: {
 from: "livros",
 localField: "_id_livro",
 foreignField: "_id",
 as: "livro"
 }
 }
)

Associar EMPRESTIMOS e USUARIO:
db.Emprestimo.aggregate([
 {
 \$lookup: {
 from: "Usuario",
 localField: "_id",
 foreignField: "_id",
 as: "usuario"
 }
 }
)

```

    { $unwind: "$usuario" },
    { $project: { _id: 0, emprestimo: "$$ROOT", usuario: 1 } }
  ])

```

9. Exemplos de Aggregate – Loja de Departamentos

Associar VENDAS e PRODUTOS:

```

db.Vendas.aggregate([
  {
    $lookup: {
      from: "Produtos",
      localField: "_id",
      foreignField: "_id",
      as: "produto"
    }
  },
  { $unwind: "$produto" },
  { $project: { _id: 0, venda: "$$ROOT", produto: 1 } }
])

```

Associar VENDAS e CLIENTE:

```

db.Vendas.aggregate([
  {
    $lookup: {
      from: "Cliente",
      localField: "_id",
      foreignField: "_id",
      as: "cliente"
    }
  },
  { $unwind: "$cliente" },
  { $project: { _id: 0, cliente: 1, vendas: 1 } }
])

```

10. Questões de Revisão Teórica

- Relacional x NoSQL: esquema rígido x flexível.
- ACID: Atomicidade, Consistência, Isolamento e Durabilidade.
- CAP: Consistência, Disponibilidade e Tolerância a Partições.
- Cluster: vários servidores funcionando como um.
- MongoDB: orientado a documentos, JSON/BSON, escalável.
- Cassandra: orientado a colunas, alta tolerância a falhas.
- Neo4J: orientado a grafos, linguagem Cypher.
- Redis: chave-valor em memória, muito rápido.